Distributed mutual exclusion

Contents

Problem definition

- N processes share a resource that can be accessed by one process at a certain time
- The process accesing the critical resource is said to be in the critical section (CS)
- Conditions to be satisfied:
  - Safety: only one process can acces the shared resource at a time
  - Liveness: if a process requests to access the shared resource it should eventually be given a chance to do so

Why distributed mutual exclusion is important?

- Basically, the same motivation as for non-distributed mutual exclusion
- Exclusive access of any shared resources
  - N users want to print to a shared printer

System model

- Distributed system consisting of a collection of N processes P_i, i=1, 2, …, N
- Each process executes on a single processor
- Processes communicates only by message passing
- Each process P_i has a state s_i
- Each process executes a sequence of events
- There are three types of events: send, receive and operation that transforms the state
- There is no global time

Distributed mutual exclusion algorithms

- Used in an environment where the communication is done by message passing
- Token-based algorithms
  - Token-ring
  - Suzuki-Kasami
- Non-token based algorithms
  - Central coordinator
  - Lamport
  - Ricart-Agrawala

Token based mutual exclusion algorithms

Idea:

- a unique token moves among the processors;
- a processor can enter the CS only if it has the token

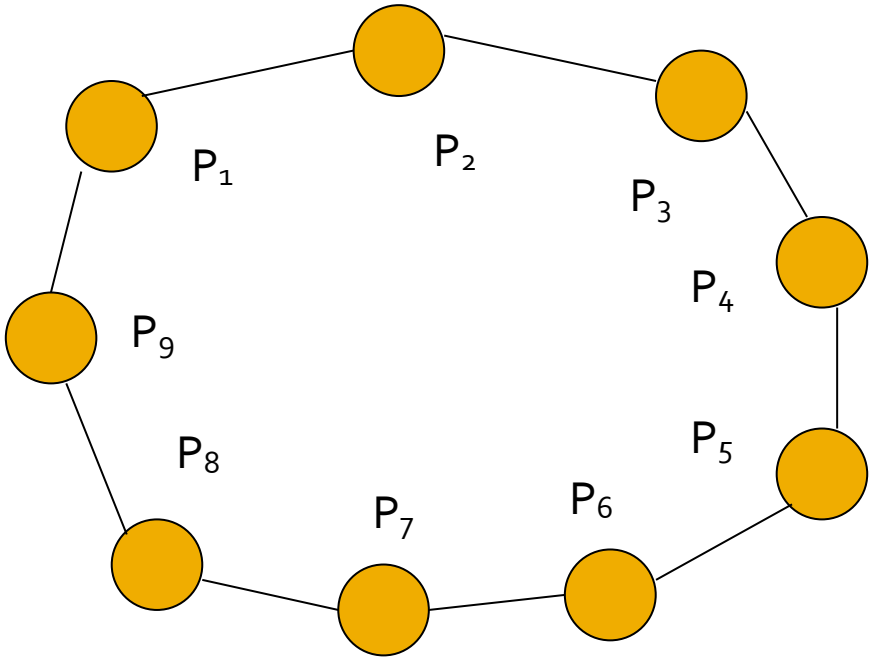
Algorithms:

Token ring

Suzuki-Kasami

Token ring algorithm

- Processors are aranged in a logical ring
- First processor P_1 gets token



Token ring algorithm

- When process P_i gets the token, it checks if it wants to enter CS
  - If it wants to enter CS, simply it enters CS
  - If it does not want to enter CS, it sends the token to $P_{(i+1) \bmod N}$
- Properties
  - Simple algorithm
  - Suitable for small N
  - Problem with missing token

Suzuki-Kasami's algorithm

- $P_1, P_2, \ldots, P_n$ processors
- Each processor P_i holds a timestamp vector T
  - $T_i[j]$ is the timestamp of the most current request from P_j known by P_i
- Token holds a vector X
  - X[i] is the timestamp of the last request from P_i that has been served
- One can easily find if a processor has requests that have not been yet served by comparing T and X

Suzuki-Kasami's algorithm

- **P_i requests CS**
  - P_i increase T_i [i] by 1 and broadcasts *request* (T_i *[i], i)* to all processors
  - When P_j receives the *request* (T_i *[i], i)*
    - Update T_j [i] to max (T_j [i], T_i [i])
    - If it does not have the token does nothing
    - If it has the token and does not need it, then it releases the token

Suzuki-Kasami's algorithm

- **P_j releases the CS**
  - P_j sets $X[j] = T_j[j]$
  - Check T_j starting with $T_j[j+1]$
  - If for some k: $T_j[k] > X[k]$ then P_k has a waiting request, send the token to P_k
- **P_i enters the CS**
  - If P_i has the token

Suzuki-Kasami's algorithm

- Requires N messages per request
- Problem with missing token

Non-token based mutual exclusion algorithms

Idea:

- When a processor wants to enter the critical section, it sends a timestamped message to the other processors.
- In case there are many nodes that try to enter the critical section the one with the lowest timestamp wins.

Algorithms:

Central coordinator (it does not obey the above idea)
Lamport
Ricart-Agrawala

Central coordinator algorithm

- P_1, P_2, …, P_n processors,
- C central coordinator – grants access to CS

- **P_i requests CS**
  - P_i sends a *request* to C and waits for a reply

- **P_i enters the CS**
  - when it gets the *reply* from coordinator

- **P_i releases the CS**
  - notifies the coordinator with a *release* message

Central coordinator algorithm

- Properties
  - simple
  - only three messages per CS use
  - Requests are granted in order in which they are received
- Problems
  - Coordinator – performance bottleneck
  - Coordinator – point of failure

Lamport's algorithm

- P_1, P_2, …, P_n processors
- Each processor has a queue where mutual exclusion requests are kept, the requests are ordered by their timestamps
- Each processor maintains a Lamport logical clock
- Messages are delivered in FIFO order between every pair of processors

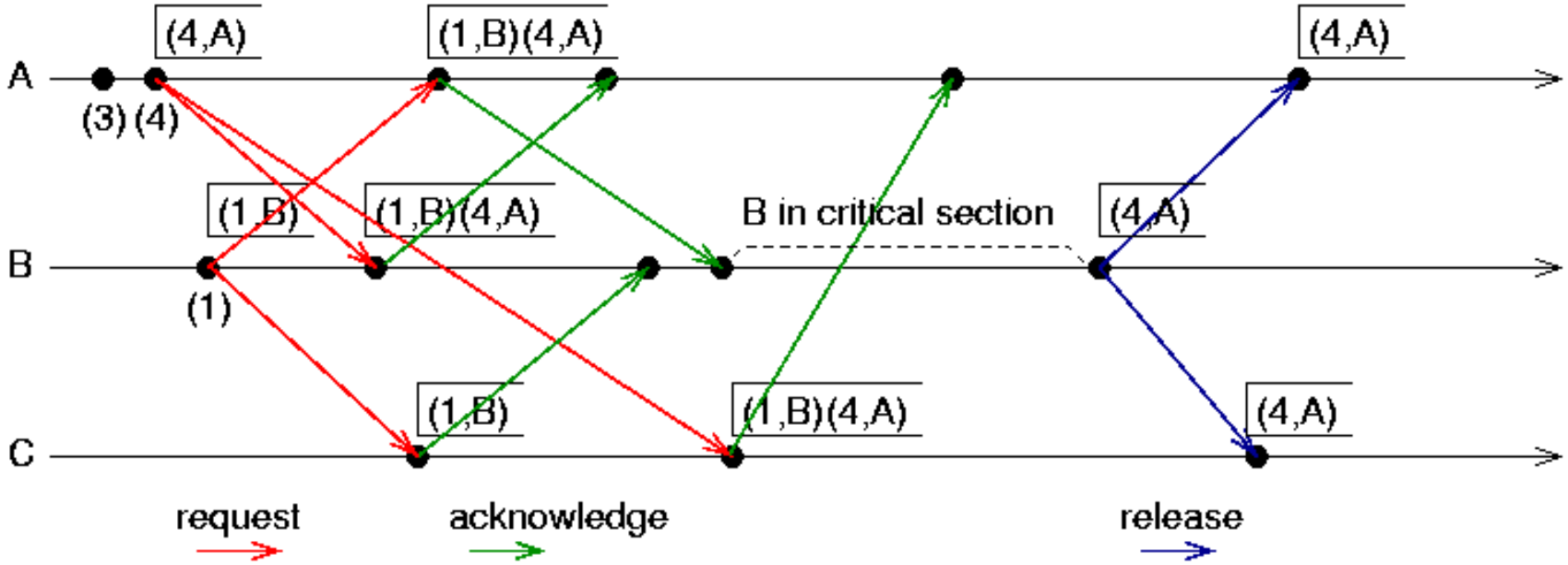
Lamport's algorithm

- **P_i requests CS**
  - P_i broadcasts the request *request(i, LC_i)* to all processors
  - P_i puts the request in its own queue (in the order of the timestamps of the requests)
  - When P_j receives a request, it returns a timestamped reply *reply(j, LC_j)* to P_i
  - P_j puts the request in its own queue

Lamport's algorithm

- **P_i enters the CS**
  - If P_i has received (reply) messages from all processors with timestamps larger than its own and its request is at the top of its queue

Lamport's algorithm



Lamport's algorithm

- **P_i releases the CS**
  - Upon exiting CS, P_i removes its request from the request queue and sends a timestamped release *release(LC_i)* message to all processors
  - When P_j receives a release message it deletes the request from its queue (this way its own request can get to the top of the queue)

Lamport's algorithm

- The algorithm requires:
  - Total ordering of events
  - All processors to be alive
- Performance
  - 3(N-1) messages per request

Ricart-Agrawala's algorithm

- P_1, P_2, …, P_n processors
- No request queue
- Each processor maintains a Lamport logical clock
- Does not need FIFO channels
- Optimization of Lamports's algorithm as the *release* messages are merged with the *reply* messages

Ricart-Agrawala's algorithm

- **P_i requests CS**
  - P_i broadcast the request *request(i, LC_i)* to all processors
  - When P_j receives a request
    - If it has not requested or it is not executing a CS, then it returns a timestamped reply *reply(j, LC_j)* to P_i
    - If it has requested a CS but $LC_j > LC_i$, then it returns a timestamped reply *reply(j, LC_j)* to P_i
    - Otherwise, postpones the reply

Ricart-Agrawala's algorithm

- **P_i enters the CS**
  - If P_i has received reply messages from all processors
- **P_i releases the CS**
  - Upon exiting CS, P_i sends *reply(i, LC_i)* to all the postponed requests

Ricart-Agrawala's algorithm

- The algorithm requires:
  - Total ordering of events
  - All processors to be alive
- Performance
  - 2(N-1) messages per request